Taller 8: Sistemas de Tiempo Real con Zephyr RTOS

José Bernardo Barquero Bonilla
2023150476

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Curso: CE1113 - Sistemas Empotrados

Prof. Dr.-Ing Jeferson González Gómez

12 de Noviembre de 2026

Parte 1: Expropiación

Las pruebas corren sobre `qemu_riscv64` (Zephyr v3.7 LTS). Cada variante se compiló en un directorio propio (`west build -d`) con macros pasadas por `EXTRA_CFLAGS`; sin ellas, el código reproduce la base del enunciado (comandos en `README.txt`).

R1. Traza del firmware base

Mientras BAJA (prioridad 5) ejecuta su espera activa de 1.5 s ($t = 0$ a 1500 ms), ALTA (prioridad 1) imprime siete veces y MEDIA (prioridad 3) dos veces (Figura 1): cada vez que un hilo de mayor prioridad despierta de `k_msleep()`, el kernel expropia a BAJA de inmediato. Los periodos de 210 y 510 ms (en lugar de 200 y 500 ms) se deben a que `k_msleep()` garantiza un tiempo mínimo y redondea al tick del sistema (10 ms).

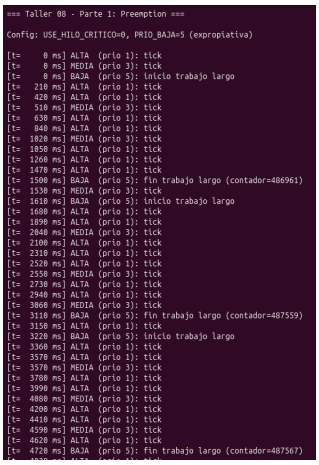


Figura 1: R1: firmware base (6.7 s). ALTA y MEDIA se intercalan durante el trabajo largo de BAJA.

R2. Cuarto hilo de mayor prioridad

Se agregó el hilo CRIT con prioridad 0 (mayor que `PRIO_ALTA`= 1), creado en `main()` con `k_thread_create(..., PRIO_CRITICA, 0, K_NO_WAIT)`:

```
static void hilo_critica_fn(void *p1, void *p2, void *p3)
{
    while (1) {
        printk("[t=%6u ms] CRIT  (prio %d): tick\n",
               (unsigned int)k_uptime_get(), PRIO_CRITICA);
        k_msleep(50);
    }
}
```

En la Figura 2, CRIT aparece cada 60 ms y domina la traza. Cuando despierta en el mismo tick que otro hilo, CRIT imprime primero ($t = 420$, 840 y 1260 ms antes que ALTA; $t = 1020$ ms antes que MEDIA), porque el calendarizador siempre elige el hilo listo de mayor prioridad. El orden relativo de ALTA, MEDIA y BAJA no cambia. BAJA termina igual

en $t = 1500$ ms (su espera se mide por reloj), pero su contador baja de 486 961 a 485 344: CRIT le quita tiempo de CPU.

R3. Prioridad cooperativa en BAJA

Con `PRIO_BAJA`$= -1$ (cooperativa) y CRIT activo (Figura 3): BAJA imprime primero aunque se crea al final, porque su prioridad supera la de `main()` (0) y lo expropia en el propio `k_thread_create()`. Entre "inicio" y "fin" del trabajo largo (0–1500, 1610–3110, 3220–4720 ms) no aparece *ninguna* otra línea. Al terminar, CRIT, ALTA y MEDIA imprimen en el mismo milisegundo (1500, 3110, 4720 ms), y el periodo efectivo de ALTA pasa de 210 a 1610 ms.

Un hilo cooperativo no puede ser expropiado por ningún hilo expropiativo, sin importar su prioridad. La interrupción del temporizador sigue ocurriendo y marca a CRIT, ALTA y MEDIA como listos, pero el kernel no cambia de contexto mientras el hilo actual sea cooperativo. El cambio solo ocurre cuando BAJA cede el CPU voluntariamente con `k_msleep(100)`: en esos 100 ms se atiende la cola de listos (la ráfaga) y, al despertar, BAJA vuelve a tomar el CPU.

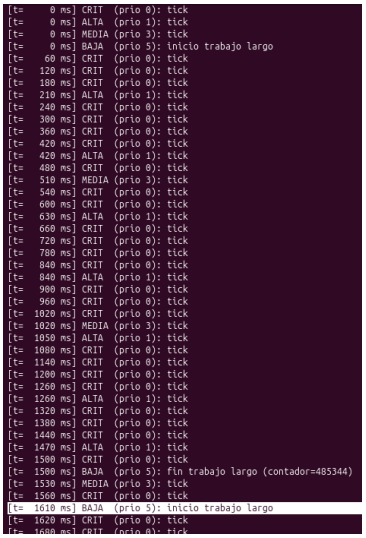


Figura 2: R2: con CRIT (prioridad 0, cada 50 ms).

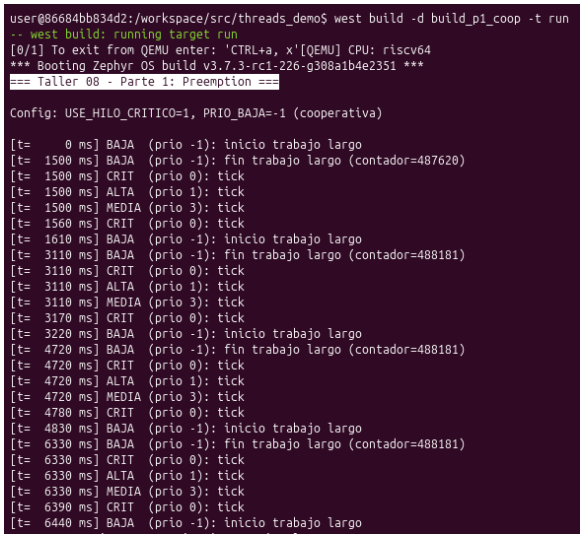


Figura 3: R3: BAJA cooperativa (-1). Huecos de 1.5 s y ráfagas al ceder el CPU.

Parte 2: Inversión de prioridades

R4. Latencia de ALTA

Latencia medida entre "ALTA: solicitando recurso" y "ALTA: recurso obtenido" (Figuras 4 y 5):

Recurso	Solicitud (ms)	Obtención (ms)	Latencia (ms)
Semáforo (`USE_MUTEX 0`)	310	1850	**1540**
Mutex (`USE_MUTEX 1`)	310	1210	**900**

Tabla 1: Latencia de ALTA según las trazas seriales.

```
*** Booting Zephyr OS build v3.7.3-rc1-226-g308a1b4e2351 ***
=== Taller 08 - Parte 2: Inversion de prioridades (USE_MUTEX=0) ===

[t=     0 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=   150 ms] BAJA : tomando el recurso
[t=   150 ms] BAJA : [R6] prioridad ANTES de tomar el recurso = 5
[t=   150 ms] BAJA : recurso tomado, iniciando seccion critica larga (prioridad efectiva=5)
[t=   150 ms] BAJA : [R6] ronda 0, prioridad efectiva = 5
[t=   170 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=   310 ms] ALTA : solicitando recurso...
[t=   320 ms] BAJA : [R6] ronda 1, prioridad efectiva = 5
[t=   340 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=   490 ms] BAJA : [R6] ronda 2, prioridad efectiva = 5
[t=   510 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=   660 ms] BAJA : [R6] ronda 3, prioridad efectiva = 5
[t=   680 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=   830 ms] BAJA : [R6] ronda 4, prioridad efectiva = 5
[t=   850 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1000 ms] BAJA : [R6] ronda 5, prioridad efectiva = 5
[t=  1020 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1170 ms] BAJA : [R6] ronda 6, prioridad efectiva = 5
[t=  1190 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1340 ms] BAJA : [R6] ronda 7, prioridad efectiva = 5
[t=  1360 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1510 ms] BAJA : [R6] ronda 8, prioridad efectiva = 5
[t=  1530 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1680 ms] BAJA : [R6] ronda 9, prioridad efectiva = 5
[t=  1700 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1850 ms] BAJA : fin seccion critica, liberando recurso
[t=  1850 ms] ALTA : recurso obtenido
[t=  1850 ms] ALTA : recurso liberado, fin
[t=  1850 ms] BAJA : [R6] prioridad DESPUES de liberar el recurso = 5
[t=  1870 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  2040 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  2210 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  2380 ms] MEDIA: trabajo de CPU (no usa el recurso)
```

Figura 4: R4/R6: semáforo. MEDIA sigue ejecutando mientras ALTA espera.

```
=== Taller 08 - Parte 2: Inversion de prioridades (USE_MUTEX=1) ===

[t=     0 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=   150 ms] BAJA : tomando el recurso
[t=   150 ms] BAJA : [R6] prioridad ANTES de tomar el recurso = 5
[t=   150 ms] BAJA : recurso tomado, iniciando seccion critica larga (prioridad efectiva=5)
[t=   150 ms] BAJA : [R6] ronda 0, prioridad efectiva = 5
[t=   170 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=   310 ms] ALTA : solicitando recurso...
[t=   310 ms] BAJA : [R6] ronda 1, prioridad efectiva = 1
[t=   410 ms] BAJA : [R6] ronda 2, prioridad efectiva = 1
[t=   510 ms] BAJA : [R6] ronda 3, prioridad efectiva = 1
[t=   610 ms] BAJA : [R6] ronda 4, prioridad efectiva = 1
[t=   710 ms] BAJA : [R6] ronda 5, prioridad efectiva = 1
[t=   810 ms] BAJA : [R6] ronda 6, prioridad efectiva = 1
[t=   910 ms] BAJA : [R6] ronda 7, prioridad efectiva = 1
[t=  1010 ms] BAJA : [R6] ronda 8, prioridad efectiva = 1
[t=  1110 ms] BAJA : [R6] ronda 9, prioridad efectiva = 1
[t=  1210 ms] BAJA : fin seccion critica, liberando recurso
[t=  1210 ms] ALTA : recurso obtenido
[t=  1210 ms] ALTA : recurso liberado, fin
[t=  1210 ms] BAJA : [R6] prioridad DESPUES de liberar el recurso = 5
[t=  1230 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1400 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1570 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1740 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  1910 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  2080 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  2250 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  2420 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  2590 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  2760 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  2930 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  3100 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  3270 ms] MEDIA: trabajo de CPU (no usa el recurso)
[t=  3440 ms] MEDIA: trabajo de CPU (no usa el recurso)
```

Figura 5: R4/R6: mutex. Sin líneas de MEDIA mientras ALTA espera.

R5. Explicación de la diferencia

Semáforo: `k_sem` no tiene dueño ni herencia de prioridad. Con ALTA bloqueada desde $t = 310$ ms, BAJA conserva su prioridad 5, menor que la de MEDIA (3). MEDIA, que no usa el recurso, alterna 150 ms de CPU con 10 ms de sueño y expropia a BAJA en cada ciclo: las rondas de BAJA se espacian 170 ms (320, 490, ..., 1680 ms) y MEDIA imprime nueve veces mientras ALTA espera. ALTA queda bloqueada indirectamente por un hilo con el que no interactúa (inversión de prioridades).

Mutex: `k_mutex` registra a BAJA como dueña. Cuando ALTA se bloquea en `k_mutex_lock()`, el kernel aplica *herencia de prioridad*: BAJA hereda temporalmente la prioridad 1 de ALTA, MEDIA ya no puede expropiarla, las nueve rondas restantes corren seguidas cada 100 ms (310–1110 ms) y MEDIA no imprime en ese intervalo. En `k_mutex_unlock()`, BAJA recupera su prioridad 5 y ALTA obtiene el mutex de inmediato: su espera queda acotada a la sección crítica de BAJA. La diferencia no es mayor porque `busy_wait_ms()` termina por tiempo de reloj y no por trabajo realizado.

R6. Prioridad efectiva de BAJA

En BAJA se imprimió `k_thread_priority_get(k_current_get())` antes de tomar el recurso, al inicio de cada ronda de la sección crítica y después de liberarlo (líneas `[R6]`). En modo mutex (Figura 5) la prioridad es **5** antes de tomar el recurso y en la ronda 0 ($t = 150$ ms, antes de la solicitud de ALTA). Pasa a **1** desde la ronda 1 ($t = 310$ ms, justo

cuando ALTA se bloquea) hasta la ronda 9, y regresa a **5** al liberar el recurso ($t = 1210$ ms). Con semáforo (Figura 4) permanece en 5 todo el tiempo.

Parte 3 (opcional): SMP

R7. Ejecución con SMP y asignación de hilos a harts

Diagnóstico del intento. Cada línea SMP incluye `[cpu N]`. Al agregar `CONFIG_SMP=y` y `CONFIG_MP_MAX_NUM_CPUS=4` a `qemu_riscv64`, el firmware compila y QEMU se lanza con `-smp cpus=4`, pero la ejecución falla: solo CRIT imprime, por primera vez en $t = 1400$ ms y cada 200 ms en vez de 50 ms (4×, igual al número de harts). ALTA, MEDIA y BAJA no ejecutan en 20 s (no se determinó la causa exacta de esto último). El defconfig de la variante oficial `qemu_riscv64/qemu_virt_riscv64/smp` (`SMP=y`, `MP_MAX_NUM_CPUS=2`, `QEMU_ICOUNT=n`) señaló la causa: con `CONFIG_QEMU_ICOUNT`, QEMU deriva el tiempo virtual del conteo de instrucciones, que se reparte entre los harts. Recompilar el mismo intento con `-DCONFIG_QEMU_ICOUNT=n` da una ejecución exitosa con 4 harts:

```
[SMP] CONFIG_MP_MAX_NUM_CPUS=4, USE_AFINIDAD=0, main en cpu 0
[t=    10 ms] [cpu 3] CRIT  (prio 0): tick
[t=    10 ms] [cpu 1] ALTA  (prio 1): tick
[t=    10 ms] [cpu 0] BAJA  (prio 5): inicio trabajo largo
[t=  1630 ms] [cpu 2] BAJA  (prio 5): inicio trabajo largo
[t=  3130 ms] [cpu 0] BAJA  (prio 5): fin trabajo largo (contador=1447527)
```

Asignación de hilos a harts. Zephyr no asigna hilos de forma fija: cada hart, al recalendarizar, toma de una cola global el hilo listo de mayor prioridad que no esté corriendo en otro hart. Los hilos migran, incluso a mitad de ejecución: BAJA inicia en el hart 2 ($t = 1630$ ms) y termina en el hart 0 ($t = 3130$ ms).

Afinidad. Sí es posible fijar un hilo a un hart, con la opción `CONFIG_SCHED_CPU_MASK=y` y la función `k_thread_cpu_pin()`, pero solo con el hilo detenido. Con `USE_AFINIDAD=1`, los hilos se crean con `K_FOREVER`, se fijan al hart 0 y se arrancan con `k_thread_start()`. Las llamadas devolvieron 0 y la traza muestra solo `[cpu 0]`.

Experimento	1 hart	SMP (2 harts)	SMP + afinidad a hart 0
BAJA cooperativa: huecos sin otros hilos	1500 ms	ninguno	1500 ms
Latencia de ALTA, semáforo	1540 ms	690 ms	1550 ms
Latencia de ALTA, mutex	900 ms	700 ms	–

Tabla 2: Partes 1 y 2 con uno y dos harts (variante oficial SMP).

Con dos harts, el hilo cooperativo solo monopoliza su hart y la inversión desaparece incluso con semáforo: MEDIA y BAJA ya no compiten por el mismo procesador. Al fijar todo al hart 0, ambos efectos reaparecen.